

Program: BTech – Computer Science and Engineering

Course Name: Computer Organization and Architecture

(Course Code: BTCSE022401)



Course Conducted

By

Mrs. Ashwini N. Sawant

Assistant Professor of CSE,

Department of Computer Science and Engineering,

School of Engineering and Technology,

D. Y. Patil Agriculture and Technical University, Talsande, Kolhapur.

Unit 3

Computer Arithmetic

Introduction to Computer Arithmetic

- **Computer arithmetic** is the study of how arithmetic operations are performed inside a digital computer using **binary numbers**.
- computers operate only on **0s and 1s**, so all arithmetic must be converted into binary form and implemented using digital logic circuits.
- Computer arithmetic is a critical part of **(COA)** because it directly

1. Arithmetic Operations

These operations involve numerical calculations.: Common Arithmetic Functions:

- **Addition** $\rightarrow A + B$
- **Subtraction** $\rightarrow A - B$
- **Multiplication Division**
- **Increment** $\rightarrow A + 1$
- **Decrement** $\rightarrow A - 1$
- **Comparison** (greater than, less than, equal)

Binary Addition Rule :

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Binary Subtraction Rules

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Decimal	Binary	Decimal	Binary
0	0	12	1100
1	1	13	1101
2	10	14	1110
3	11	15	1111
4	100	16	10000
5	101	17	10001
6	110	18	10010
7	111	19	10011
8	1000	20	10100
9	1001	21	10101
10	1010	22	10110
11	1011	23	10111

1's Complement (One's Complement)

Definition - **1's complement** of a binary number is obtained by **inverting all bits**:

- $0 \rightarrow 1$
- $1 \rightarrow 0$

Example : If $A = 101100$ then

One's Complement of A is $= 010011$

2's Complement (Two's Complement)

Definition

2's complement is obtained by: Rule

1. Taking **1's complement**
2. Adding **1** to it

Subtraction Using 1's Complement

Rules:

- 1) Assume first no. is **A** & second no. is **B** , write their binary equivalents.
- 2) Find **1's complement** of second no. (**B bar**)
- 3) Do addition of first no. with **1's complement** of second no. (i.e $A + \bar{B}$)
- 4) If carry is present in addition, then answer is +ve and add carry into answer (i.e end around carry)
- 5) If carry is absent then answer is -ve and find **1's complement of answer.**

Subtraction Using 2's Complement

Rules:

- 1) Assume first no. is **A** & second no. is **B** , write their binary equivalents.
- 2) Find **2's complement** of second no. (**$B' = B^- + 1$**)
- 3) Do addition of first no. with **2's complement** of second no. (i.e **$A + B'$**)
- 4) If carry is present in addition, then answer is **+ve** and ignore the carry
- 5) If carry is absent then answer is **-ve** and find **2's complement of answer.**

Floating Point Numbers: Normalization -

It is the process of representing a floating-point number in a standard scientific form so that there is **only one non-zero digit before the decimal point**

What is Normalization

A floating-point number is generally represented as:

$$\text{Number} = \text{Mantissa} \times \text{Base}^{\text{Exponent}}$$

- Required in **IEEE 754 floating-point format** in the base is **2 (binary)**.

A number is **normalized** when:

- **In decimal** → Only one non-zero digit before decimal

Example:

$$345.6 = 3.456 \times 10^2 \checkmark \text{ (normalized)}$$

- **In binary** → Only one 1 before binary point

Example:

$$1011.01_2 = 1.01101 \times 2^3 \checkmark \text{ (normalized)}$$

Why Normalization is Important ;

- Normalization means writing a floating-point number so that **only one non-zero digit (1 in binary)** comes before the decimal point.
- Provides Unique Representation-Normalization ensures **only one standard form**, so every number has **one unique representation**
- **Maximizes precision** means:
It allows a number to store **maximum possible accurate digits** without wasting space.
- Precision in Floating-Point Numbers

The **mantissa** stores the important digits of the number.

More mantissa bits → more digits stored → **more precise result**

Fewer mantissa bits → fewer digits stored → **rounding errors**

3.141592 If mantissa allows **only 3 digits**: → **3.14** (**some digits lost ✗**)

If mantissa allow digits: → **3.141592** (**more accurate ✓**)

IEEE 754 Floating-Point Representation

- Institute of Electrical and Electronics Engineers.
- Since its adoption in 1985, IEEE 754 has been the foundation for nearly all modern processors and programming languages.
- IEEE 754 is the standard used by computers to store **real (decimal) numbers** in binary form.
- It represents numbers in **scientific binary form**:

$$(-1)^{Sign} \times 1.Mantissa \times 2^{(Exponent - Bias)}$$

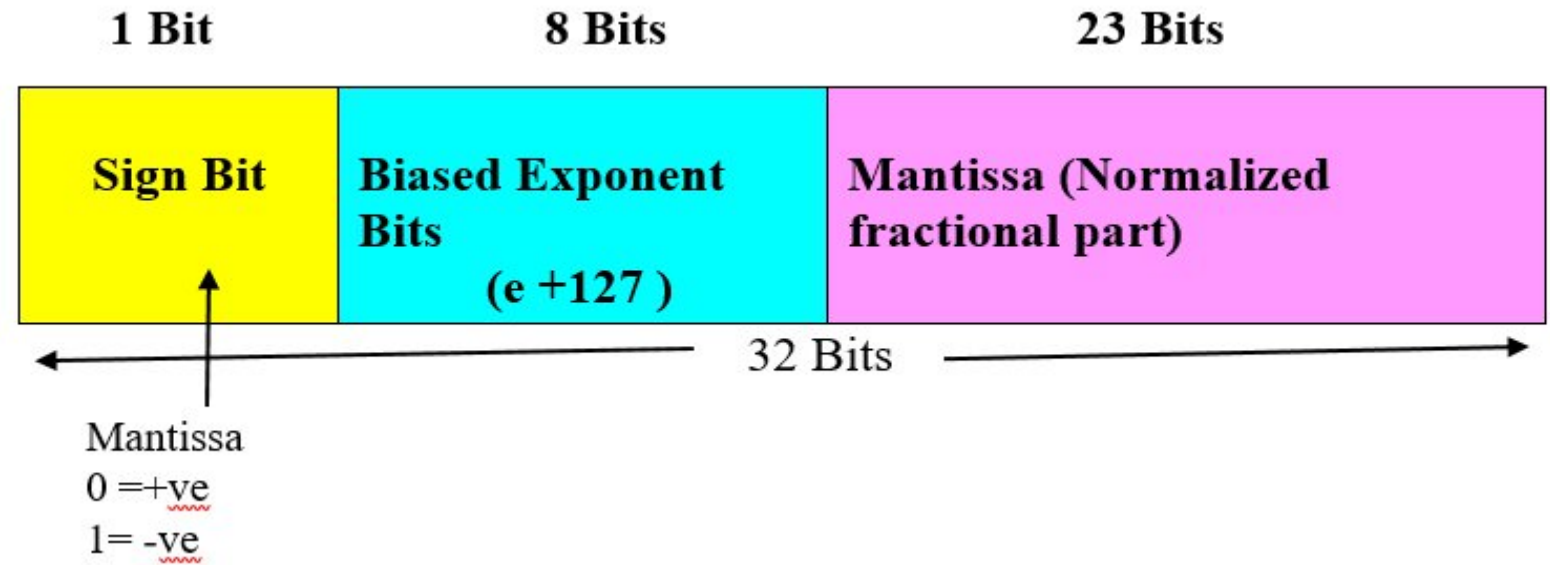
A floating-point number has **3 parts**:

Part	Purpose
Sign bit (S)	0 = positive, 1 = negative
Exponent (E)	Stores power of 2
Mantissa (M)	Stores significant digits (fraction part)

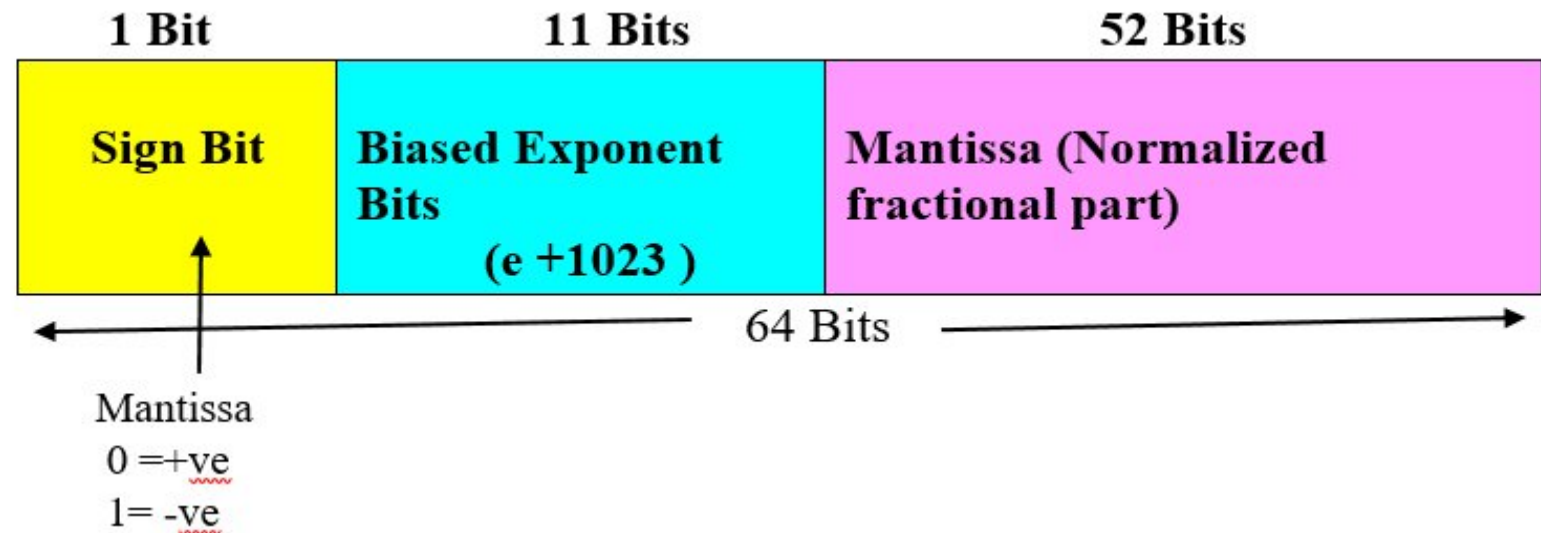
IEEE 754 Floating-Point Representation:

In two ways

1) Single Precision Format



2) Double Precision Format



Represent number in Single & double precision floating point format

Step 1 : Convert given no. in binary

Step 2 : Represent binary number in normalized form

Step 3: IEEE 754 floating point in

i) Single precision floating point format:

Bias for single precision floating point format is 127

Biased exponent $=E'=E+127$

ii) Double precision floating point format

Bias for single precision floating point format is 1023

Biased exponent $=E'=E+1023$

Example: Represent $(178.1875)_{10}$ in Single & double precision floating point format

Floating-Point Arithmetic

Where:

- **S** = Sign bit (0 = positive, 1 = negative)
- **M** = Mantissa (fraction part)
- **E** = Exponent

Rules for Floating-Point Multiplication

Step1: Sign

Step2 : Add Exponent

Step 3 : Multiply the mantissa

Step 4: Normalize the result